Existential Quantification in Boolean Logic using Factor Graphs

Dissertation

Submitted in partial fulfillment of the requirements
for the degree of

Master of Technology

by

Parakram Majumdar

Roll No: 05D05004

under the guidance of

Dr. Supratik Chakraborty

Department of Computer Science and Engineering
Indian Institute of Technology, Bombay
Mumbai

Abstract

Quantified Boolean Formulae are a generalization of SAT, which is a well known NP-complete problem, and since universally quantifying out a variable from a boolean formula in CNF is trivial, existential quantification is really the hard part. Interestingly, existential quantification of a variable from a conjunction of functions falls into the exact domain of Factor Graph algorithms. The purpose of the project was to explore *the application of factor graph algorithms in computing the and-exists operator* (existential quantification on a conjunction of functions). This document summarizes both the theoretical results, as well as an implementation of factor graphs. We also explore the application of the factor graph method in two problems : *Symbolic Model Checking* for formal verification of Hardware, and in *Quantified Boolean Formulae*.

Symbolic Model Checking is an approach to formal verification of digital circuit designs. Each wire in the circuit is represented by a boolean variable, and hence, a zero order formula represents a set of states of the circuit. Transitions from current state(s) to the next is encoded in the form of another formula and computing the set of reachable states boils down to conjunction, disjunction, and existential quantification of boolean formulas. We prove that applying the factor graph method to existential quantification allows us to trade peak memory requirements with accuracy. Interestingly, even if an inaccurate result is obtained, the result is always implied by the actual result. In other words, *the approximate set of states computed using the factor-graph algorithm is always an over-approximation of the actual set of states.* Thus, if the set of reachable states computed using our methods does not contain any erroneous or undesirable states, then the actual set of reachable states also does not contain any undesirable states.

For computing the result accurately, we need to make the factor graph acyclic, which amounts to either merging nodes in the graph and hence increasing the peak memory requirement, or splitting the problem into two by assigning true and false to a variable. This splitting on one or more variables requires a lot of book-keeping if we are not to disregard the quantifier ordering. We propose a data structure called the *Split Record Table*, and summarize an implementation of the same.

Acknowledgements

I offer my humble obeisances to The Supreme Lord, Rishikesha, Lord Sri Krsna.

A sincere thanks to Professor Supratik for guiding me through this project, for the freedom he gave me for working on various avenues, and for always being there whenever I requested a meeting or a discussion.

I would like to acknowledge Annervaz's contributions to this work from the various discussions that we had, for walking me through his FRAIG implementation, and for sharing his report with me. A hearty thanks to Nisarg Shah and Rahul Srinivasan for working with me and injecting some of their high adrenaline enthusiasm into this project.

I would like to thank my family : my parents and my grand mother, for being so supportive and understanding, and especially my sister, for keeping me in high spirits with her cheerful company, as I wrote this report. I would also like to thank Prof. Ganesh Ramakrishnan, and Sohinee Ganguly for giving invaluable suggestions in writing a report.

-Parakram Majumdar

Contents

1 Introduction **3**

1.1 And-exists operator in symbolic model checking . . . 3

1.2 Quantified Boolean Formulas . . . 4

1.3 Sum Product Algorithm . . . 5

1.4 Cudd . . . 8

1.5 Related Work . . . 9

2 Theoretical results **10**

2.1 The Message Passing algorithm on the And-Exists operator . . . 10

2.2 Termination of the algorithm . . . 11

2.3 Over Approximation . . . 12

2.4 Comparison with distribution over conjunction . . . 13

2.5 Projection over multiple variables . . . 17

2.6 Leveraging quantifier scheduling . . . 18

2.7 Reusing of messages . . . 19

3 Factor Graph Implementation **22**

3.1 Factor Graph . . . 22

3.2 Structs . . . 22

3.3 Making the graph acyclic . . . 23

3.4 Checkpoints . . . 24

3.5 Convergence . . . 24

4 Split Record Table **26**

4.1 SRT definitions . . . 26

4.2 QBF_solve . . . 28

4.3 Merging of nodes . . . 31

4.4 Deductions . . . 32

5 Experiments **36**

6 Future work **40**

Chapter 1

Introduction

The and-exists operator existentially quantifies a given set of boolean variables from the conjunction of a given set of boolean functions. Sections 1.1 and 1.2 describe two areas where the and-exists operator finds application.

Factor graphs (Kschischang et al. [2001]) find their applications in many branches of computer science including artificial intelligence, signal processing and digital telecommunications. Speaking at a very high level, factor graphs depict the inter-dependencies between various elements of a problem, and allow us to find an efficient sequence of computations in order to get to the result. Factor graphs lend smooth application to aggregation of a product of functions in semi-rings, and hence are applicable to the and-exists operator. The Sum-Product operator in marginal probability calculation is an analogous problem to the And-exists operator. Section 1.3 provides a quick description of the sum product algorithm, which is a specific application of factor graphs, to build intuition useful for understanding the later chapters.

1.1 And-exists operator in symbolic model checking

Symbolic model checking involves representing the circuit in propositional logic. For each wire, we assign a variable that represents whether the signal in the wire is high or low. Let the set of such variables be V. Thus, any boolean formula over these variables is a set of possible states over V. Let $S(V)$ be a set of states at a particular moment. Now, depending on how the wires are connected using gates, any given state will evolve into a new state after some time. Let V' denote the set of variables that represent the signals in the *next state*. Let $T(V, V')$ be the boolean formula representing the *transition relation* from a state to the next state. Thus, T describes how the wires are connected to each other using gates. Then, the set of states $S'(V')$in the next moment, starting from $S(V)$ can be obtained as

$$S'(V') = \exists V \ [S(V) \wedge T(V, V')]$$

Symbolic model checking achieved its major breakthrough after the introduction of *Binary Decision Diagrams* (BDDs) as a data structure for representing and manipulating expressions in propositional logic [Burch et al., 1993]. However, BDD-based algorithms are known to suffer from space explosion. Hence, even BDDs are limited up to a certain size of circuits that they can handle.

Burch et al. [1993] describe a method in which this operator can be computed efficiently on BDDs, without actually having to compute $S(V) \wedge T(V, V')$. However, for large circuits, computing even $S(V)$ or $T(V, V')$ might not be possible. Instead, these BDDs can be represented as an implicit conjunction or disjunction of a set of smaller BDDs. Generalizing this operation to the case where the functions are represented as a disjunction is simple, since the existential operator distributes. However, especially for

synchronous circuits, it is more natural to represent these functions as a conjunction. Furthermore, it is sometimes useful to represent S as a conjunction of its projections S_i on different variable subsets. In this case, the operator generalizes to

$$S_i'(V') = \exists V \ [S_1(V) \wedge S_2(V) \wedge ...S_k(V) \wedge T_1(V, V') \wedge T_2(V, V') \wedge ...T_n(V, V')]$$

The BDD based method given by Burch et al. [1993] does not generalize to k and n more than 1. Instead, they break this problem into smaller problems by using the fact that typically each T_i depends only a small subset of variables. Thus, given a suitable variable ordering, they can push in the existential operators and compute smaller problems. The only drawback of course is that the user must supply the variable ordering herself. In a way, the factor graph problem takes care of this problem directly.

1.2 Quantified Boolean Formulas

A Quantified Boolean Formula, or a QBF, is a propositional logic formula with universal and existential quantifiers. Formally, a QBF can be described as

$$\begin{aligned} QBF &\rightarrow \forall v \ QBF \\ &\quad\ \ \exists v \ QBF \\ &\quad\ \ F \\ F &\rightarrow \text{propositional logic formula} \\ v &\rightarrow \text{propositional variable} \end{aligned}$$

Solving a QBF amounts to existential and universal quantification of variables from the formulae to create resultant formula. To elaborate a bit more, the resultant of $\forall v \ F$ is obtained by substituting v with both $\top$ and $\bot$ and conjoining the result, i.e., $\forall v \ F \equiv (F_{v=\bot} \wedge F_{v=\top})$. Similarly, $\exists v \ F \equiv (F_{v=\top} \vee F_{v=\bot})$. Successive application of these operators can be similarly solved.

QBF is a generalization of the SAT problem, which has only existential quantifiers. QBFs are thus harder than NP, but are solvable in P-SPACE using a brute force algorithm by keeping a recursive tree of depth lesser than or equal to the number of variables. In fact, QBFs are a canonical P-SPACE complete problem. (P-SPACE is the class of problems which can be solved in polynomial space given infinite time by a deterministic or non-deterministic Turing machine.)

The form of formulae shown above is known as the prenex normal form, wherein the quantifiers appear only at the start of the formula. Any formula can be converted to such a form by renaming the variables and moving the quantifiers outside. Also, traditionally, QBF formulae are usually posed in the CNF format (Zhang and Malik [2002]), and there is also a strong support for the claim that CNF is sufficient as an input format for SAT solvers (Järvisalo et al. [2010]).

Interestingly, the universal quantification operation on CNF formulae comes almost for free. Universal quantification of a variable from a CNF formula is equivalent to just dropping all occurrences of the variables. For example,

$$\forall z \exists x \forall y \ (x \vee \neg y) \wedge (y \vee \neg z) \wedge (\neg x \vee y \vee \neg z) \equiv \forall z \exists x \ x \wedge \neg z \wedge (\neg x \vee \neg z)$$

Thus, the existential quantification is really the hard part of solving QBFs.

1.3 Sum Product Algorithm

This section explains the conventional sum product algorithm briefly using an example. Consider the following function f that can be written as a multiple of other functions

$$f = f_1(x_1, x_2, x_3) f_2(x_3, x_4) f_3(x_4, x_5) f_4(x_4, x_6)$$

Suppose we want to calculate *the projection of* f *on* x_3, that is,

$$\sum_{\overline{x_3}} f(x_1, x_2, x_3, x_4, x_5, x_6) = \sum_{x_1, x_2, x_4, x_5, x_6} f_1(x_1, x_2, x_3) f_2(x_3, x_4) f_3(x_4, x_5) f_4(x_4, x_6)$$

One way to compute this would be to iterate over all values of the tuple $(x_1, x_2, x_4, x_5, x_6)$ and sum the valuations of f. If the size of the domain of each x_i is d_i, then the above method has time complexity $O(\Pi d_i)$. However, a better method would be as follows :

$$\left(\sum_{x_1, x_2} f_1(x_1, x_2, x_3)\right)\left(\sum_{x_4} f_2(x_3, x_4)\left(\sum_{x_5} f_3(x_4, x_5)\right)\left(\sum_{x_6} f_4(x_4, x_6)\right)\right)$$

This way, the time complexity is $O(d_3[d_1 d_2 + d_4 d_5 + d_4 d_6])$. Such a computation in which we decide upon an ordering of the variables and *push in* their quantifiers as much as possible is called *quantifier scheduling*. To understand why this schedule is the best, it is useful to look at the *factor graph* of the equation (Fig 1.1). Note how the factor graph drives the structure of the expression.

Factor graph A factor graph is a bipartite graph with the functions and variables as the two vertex sets. An edge between a *variable node* and a *function node* denotes that the variable lies in the support set of the function.

Notice that the graph is acyclic; the significance of this fact will be discussed later. *Since the graph is acyclic, we can consider it as a tree.* The optimal expression for computing the projection can be obtained by considering x_3 as the root of a tree, and computing the messages bottom up from the leaves. This way, each node receives *messages* from its children and sends a *message* to its parent.

Message A message is the result of some computation that is passed along an edge between a variable node and a function node in a factor graph. The message is always a function whose support set is the variable corresponding to the variable node.

A variable node simply multiplies all the messages coming from its children and passes the result upwards. A function node multiplies all incoming messages, multiplies itself to the result, and then projects the result on the parent variable, (i.e, sums over all variables except the parent) and passes the resultant as the message to its parent. Finally, the projection on the root can be obtained as the multiple of all incoming messages into the root node. For more insight into this algorithm, let us look at how the expression is calculated in the example above. Let $\mu_{n_1 \to n_2}$ be the message from node n_1 to node n_2. Then, the messages calculated during various iterations of the algorithm are given in Fig 1.2. Each row represents the messages computed in an iteration. Each column represents the messages in a particular branch from the root. Two columns merge when the messages from two branches merge.

Thus the algorithm, in essence, tries to push in the quantifiers as far down the tree as possible. However, notice that, if we were to compute the projection on some other variable, say x_4 in our example, we would still end up recomputing most of the messages. In other words, there is much scope of reuse of computation while computing the projections on different variables. This motivates the classic Sum Product algorithm (figure 1.3).

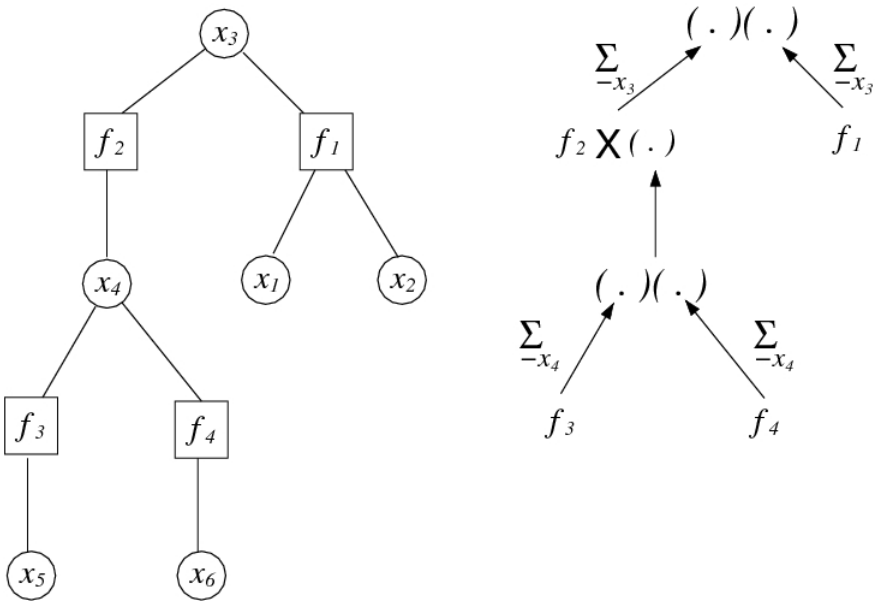

Figure 1.1: Factor graph expression tree of the given example

$\mu_{x_5 \to f_3} = 1$ $\mu_{f_3 \to x_4} = \sum_{x_5}(f_3 \times 1)$	$\mu_{x_6 \to f_4} = 1$ $\mu_{f_4 \to x_4} = \sum_{x_6}(f_4 \times 1)$	$\mu_{x_1 \to f_1} = 1$	$\mu_{x_2 \to f_1} = 1$
$\mu_{x_4 \to f_2} = (\sum_{x_5} f_3)(\sum_{x_6} f_4)$ $\mu_{f_2 \to x_3} = \sum_{x_4}[f_2 \times (\sum_{x_5} f_3) \times (\sum_{x_6} f_4)]$		$\mu_{f_1 \to x_3} = \sum_{x_1,x_2}(f_1 \times 1 \times 1)$	
Projection on $x_3 = \{\sum_{x_4}[f_2\ (\sum_{x_5} f_3)\ (\sum_{x_6} f_4)]\}\ \{\sum_{x_1,x_2} f_1\}$			

Figure 1.2: Messages being passed in the graph of Fig 1.1.

In short, instead of computing the messages from "child" to "parent", we do away with the notions of a rooted tree and a message is computed for both directions of each and every edge in the graph. Instead of starting the message passing from "leaf nodes", the algorithm is started with nodes having one neighbor each. After that, each iteration of the while loop computes the message over some edge. We call the rule for updating messages during the sum product algorithm in a general factor graph as the *message update rule*.

Message update rule The message update rule between a function node f and a variable node v is as follows:

- The message from v to f is the multiple of all messages incoming to v, other than from f. Thus

$$\mu_{v \to f} = \prod_{g \in (N_v \setminus f)} \mu_{g \to v}$$

- The message from f to v is the multiple of all messages incoming to f, other than from v, and f itself. Thus

$$\mu_{f \to v} = \sum_{\overline{v}} \left[[\prod_{u \in N_f \setminus v} \mu_{u \to f}] \times f \right]$$

where N_p stands for the set of neighbors of the node p, $\mu_{p \to q}$ is the message sent from node p to q and $\overline{v}$ is the set of all variables except v.

```
passMessages( factor_graph G ) // Assumption : G is acyclic, connected
{
   for each node n with only one neighbor in G do
      if n is a variable node then
         pass the message '1' to its neighbor;
      end if
      if n is a function node for the function f then
         pass f as the message to its neighbor v;
         //note that v is the only variable in the support set of f
         //hence, we do not need to project f on v
      end if
   end for
   while (the message over some edge is yet to be computed) do
      for each edge (n, m) s.t.  n has received msg from all nbrs other than m do
         if n is a variable node then
            μ_{n→m} = ∏_{f∈N(n)\m} μ_{f→n}
         end if
         if n is a function node then
            μ_{n→m} = ∑_{m̄} [ ∏_{v∈N(n)\m} [μ_{v→n} × n] ]
         end if
      end for
   end while
}
```

Figure 1.3: **The classic Sum Product Algorithm for computing the summary of a multiple of functions over a set of variables** *This algorithm is applicable to any semi-ring with two binary operators $+$ and $\cdot$, and the multiplicative identity* 1. *This is a dynamic programming version of the algorithm that passes all messages and computes the projection over all variables. The Message Passing algorithm for existential quantification over boolean functions can be obtained by replacing the summation by disjunction, product by conjunction, and projection by existential quantification.*

Since a message from a node is computable only if the incoming messages on all other edges of the node are available, only an edge which satisfies this condition is picked at every iteration. **In an acyclic graph**, owing to the nature of the condition given above, **each edge of the graph will be picked twice and exactly twice (once in each direction) for computing messages.** It for this reason that this algorithm will work only if the factor graph is acyclic.

Intuitively, if there are cycles in the graph, the graph cannot be seen as a tree, and pushing the summation operators to the *leaf nodes* does not make sense. If we look at the algorithm given in figure 1.3, we can see that if the graph is cyclic and has no leaf nodes, the algorithm may not even start, since each node would be just waiting for messages from "all except one node". Furthermore, in the presence of cycles, it is possible that an incoming message may have multiple possible paths from leaf nodes. This would mean that the incoming message, and also the final answer, would depend on the order in which different (m, n) pairs are evaluated. Hence it is not guaranteed whether the sum-product algorithm would give the correct results on cyclic graphs.

Also, notice that if a factor graph has a cycle, then once a message is used to compute an outgoing

message, the outgoing message may propagate its changes around the cycle and change the incoming message. There is a variant of the sum product algorithm where the update rule is applied on all nodes till convergence is reached. However, it is still not guaranteed that the algorithm will converge for a general factor graph, and that after convergence, the result obtained will be the correct one.

To make the sum product algorithm work on cyclic graphs, one approach would be to coalesce variable nodes to make a composite variable node, or to club function nodes in a similar way. However, clubbing function nodes together increases the time and memory required by the algorithm. Indeed, clubbing all the functions into a monolithic function would reduce to the brute force method. For a detailed discussion, the interested reader is referred to Kschischang et al. [2001].

Another limitation of the sum product algorithm is that it computes the projections over individual variables. To compute the projection over a set of variables, the only method known so far is to conjoin the variables into a composite variable, which, as discussed above, may make the sum product algorithm expensive.

To summarize the discussion in this section, we started by observing redundant computations while calculating the projections of a function over variables. We then discussed an algorithm that reuses these computations to efficiently compute the projection over each variable. We also briefly discussed how it is crucial for the factor graph of the expression to be acyclic, without which the algorithm does not guarantee correct answers. It was also discussed how clubbing variable nodes or function nodes together may make the graph acyclic, or may allow us to take a projection over a set of more than one variables, but such clubbing may lead to the algorithm becoming more expensive in terms of time and memory.

The message passing algorithm is a generic algorithm that works on any semi-ring, and not just the set of real numbers with the $+$ and $\cdot$ operators having their ordinary meanings. In particular, the set of boolean constants true and false, written as $\top$ and $\bot$ also constitute a semi-ring with $\vee$ (disjunction) as the summation and $\wedge$ (conjunction) as the product operator. Functions in this domain are boolean functions, and the projection of a function f on a set of variables V is given as $\exists \overline{v}\ f$. The message passing algorithm corresponding to this domain is discussed in section 2.1.

1.4 Cudd

Binary Decision Diagrams (Akers [1978]), or BDDs are a data structure we use in our implementation to represent boolean formulae. A simple BDD can be understood as a binary tree where each non-leaf node represents a variable, and a leaf node is either 1 or 0. The two branches of a non-leaf node are called the *then* branch and the *else* branch. Each BDD then refers to a "decision diagram", i.e., a diagram according to which assignment of variables leads to either 0 or 1, representing true or false. In this way, a simple BDD stores a function as the set of paths (i.e., the assignments of variables) that lead to 1. A better representation of BDDs are the Reduced Ordered BDDs, or the ROBDDs. By ordered, we mean that the variables are ordered, and all paths from the root to the constant nodes respect this order. By reduced, we mean that no sub-tree structure is repeated. Instead of two nodes pointing to different replica of the same subtree, they point to the same copy. As a result, the BDD is no longer a tree, but rather just a DAG, with each non-constant node having exactly two outgoing edges. An important property of ROBDDs is that they are canonical, i.e., each formula has a unique ROBDD. For sake of convenience, we shall refer to ROBDDs as just BDDs in the rest of the document.

A state of the art implementation of BDDs in C is the Cudd library. Cudd employs many sophisticated tricks such as a cache for all BDD operations and implicit negation of nodes, and is a very popular means to represent and manipulate boolean formulae. However, BDDs are known for intermediate memory

explosions in certain operations. In particular, the existential computation is known to choke out of memory in intermediate steps even if the initial and final BDDs are not very large. The factor graph method can be seen as an attempt to bring down the peak memory requirement by utilizing the structure inherent in the problem sets.

1.5 Related Work

Quantified Boolean Formulae are a hot research area owing to the many decision problems that can be naturally expressed in QBF. Owing to the recent success in SAT solving, most current techniques depend on DPLL-like procedures to solve QBFs, supported by a strong conflict analysis preprocessing step. There are also many resolution based procedures available, but these suffer from memory blow-ups. In contrast, DPLL based procedures avoid the memory blow up, but are much more computation intensive (Zhang and Malik [2002]).

A factor graph is a bipartite graph with functions and supporting variables as nodes, capturing the inherent dependencies and independencies of functions. Many algorithms such as the viterbi algorithm, belief propagation, the Kalman filter, and certain FFT algorithms are special instances of the factor graph algorithm (Kschischang et al. [2001]). The message passing algorithm also has a dynamic programming like advantage that it computes the summary over multiple support sets.

One approach that uses factor graphs to solve the SAT problem (existential quantification) is Survey propagation (Braunstein et al. [2005]). In this approach, the probability of a variable taking a value is computed using probabilities of other variables. This leads to a iterative message passing algorithm where each probability may be updated many times before convergence. After convergence, the most probable value among all variables and values is taken and set. Hence, survey propagation is a belief propagation algorithm, and is designed specifically keeping in mind random SAT problems close to the phase transition regions (number of clauses / number of variables ≈ 4.27). Also, being a SAT solver, it terminates on finding just one satisfying solution, whereas in QBFs, we need to find the exact formula.

To our knowledge, this is the first work done on applying factor graphs on existential quantification of a conjunction of functions.

Chapter 2

Theoretical results

2.1 The Message Passing algorithm on the And-Exists operator

In chapter 1 we saw how symbolic reachability involves writing a digital circuit in the propositional logic. In this form, a set of states, as well as the transition relation from one set of states to the next can be written as propositional logic formulae. The set of states S', reachable from an initial set of states S can be obtained as

$$S'(v') = \exists v[S(v) \wedge T(v, v')]$$

where $T(v, v')$ is the transition relation from the variables v to v'. This operation is applied repeatedly till it is established that there are no other states to be reached.

Suppose the formulae for S as well as T can be expressed as a conjunction of propositional formulae. Then, the operation obtains the form

$$S'(v') = \exists v[f_1(v_1) \wedge f_2(v_2) \wedge ... \wedge f_n(v_n)]$$

where $v_i \subset (v \cup v')$. We shall call this projection as the *And-Exists operator*.

And-Exists operator The and-exists operator is a function that takes a set of boolean functions F, and a set of variables V and computes the projection of the conjunction of F on V.

$$\texttt{and-exists}(F, V) = \exists_V \bigwedge F$$

Evidently, this operation is very similar to the sum-product operation we studied in the last chapter. In fact, it turns out that the message passing algorithm (a generalization of the sum product algorithm) works for acyclic factor graphs over any semi ring. Hence, the message passing algorithm can be used to *project the conjunction of a set of propositional logic formulae on a set of variables.*

Formally speaking, the set of boolean constants $\top$ and $\bot$ can be seen as a semi-ring with disjunction being the $+$ operator and conjunction being the $\cdot$ operator. Thus, the projection operation is the existential quantification of a boolean function over a set of boolean variables. Indeed, the sum product algorithm in figure 1.3 can be easily adapted into a message passing algorithm for this domain, by simply defining the appropriate message update rule, and initializing the messages with the additive identity $\top$.

Message update rule The message update rule between a function node f and a variable node v is as follows:

- The message from v to f is the multiple of all messages incoming to v, other than from f. Thus

$$\mu_{v\to f} = \bigwedge_{g\in(N_v\setminus f)} \mu_{g\to v}$$

- The message from f to v is the multiple of all messages incoming to f, other than from v, and f itself. Thus

$$\mu_{f\to v} = \exists_{\overline{v}} \left[[\bigwedge_{u\in N_f\setminus v} \mu_{u\to f}] \wedge f \right]$$

Here, N_p stands for the set of neighbors of the node p and $\mu_{p\to q}$ is the message sent from node p to q.

However, there are a few challenges that need to be dealt with. Firstly, the message passing algorithm need not converge when the factor graph contains cycles. To overcome this challenge, we show in the next section that since each of the variables can take only two values (or some finite number of values if it is a composite variable), the factor graph is guaranteed to converge, even if the factor graph is cyclic. In the section that follows, we show that although the projection obtained may not be exact, but it is definitely an over approximation. In other words, the set of states computed this way are a super set of the set of reachable states. This guarantee of over-approximation has an important implication when we use the message passing algorithm to check whether any undesirable set of states is reachable or not- if the final set of computed reachable states does not contain any bad states, we are certain that the system is safe.

Finally, given that the message passing algorithm may give a strict over approximation where it might be possible to get an exact answer using quantifier scheduling, a method of using quantifier scheduling in the factor-graph framework is explained.

2.2 Termination of the algorithm

Termination of the message passing algorithm on a cyclic factor graph is not guaranteed in general. This section provides a proof of convergence of the message passing algorithm for the and-exists operator on any finite graph.

Consider a factor graph $G = (F \cup X, E)$ where F is the set of functions in the expression, X is the set of variables, and $E : X \times F \to 0, 1$ is a set of edges. The message passing algorithm (MPA) passes functions in the form of messages along the edges of this graph. The message from function node f to variable node x in the tth iteration is obtained by conjoining f with the messages received at f in the previous iteration from edges other than x, and existentially quantifying out variables except x. Thus,

$$\mu^t_{f\to x} = \exists_{\overline{x}} \left(\bigwedge_{x'\in N_f\setminus x} \mu^{t-1}_{x'\to f} \wedge f \right) \tag{2.1}$$

Similarly, the message from a variable node to a function node is calculated as

$$\mu^t_{x\to f} = \exists_{\overline{x}} \left(\bigwedge_{f'\in N_x\setminus f} \mu^{t-1}_{f'\to x} \right) \tag{2.2}$$

Initially to begin with, the function *True* is sent along every edge. In subsequent iterations, the algorithm sends a function from a node u to another only if the node had received a new message at the previous iteration. Thus, the algorithm terminates once all messages are stable. Thus, this point of convergence is a fixed point of the above set of equations.

To see why the algorithm is guaranteed to terminate, consider the lattice $[\mathcal{M}, \leq, \top, \bot, \sqcap, \sqcup]$, where $\mathcal{M}$ is a set of all possible states of the algorithm, each state consisting of the functions passed along each edge of the graph at the previous iteration. The $\leq$ operator is an implication relation, where $M \leq M'$ if and only if $\forall u \forall v \ [\mu_{u\to v} \Rightarrow \mu'_{u\to v}]$. Thus, $\top$ and $\bot$ are states in which all messages are *True*, and *False*, respectively. The glb operator $\cap$ on a set of states conjoins their corresponding messages and, the lub operator, disjoins.

The rules 2.1 and 2.2 can be seen as a function $A : \mathcal{M} \to \mathcal{M}$ which operates on an element of the lattice, and outputs another element.

Let two states M and M' be such that $M \leq M'$, i.e., $\forall p \forall q [\mu_{p\to q} \leq \mu'_{p\to q}]$. Let the messages in the next iterations be $\nu_{p\to q}$ and $\nu'_{p\to q}$. Then for each function f,

$$\begin{aligned}
\nu_{f\to v} &= \exists_{\overline{v}}[(\bigwedge_{u\in N(f)\setminus v} \mu_{f\to v}) \wedge f] \\
&\leq \exists_{\overline{v}}[(\bigwedge_{u\in N(f)\setminus v} \mu'_{f\to v}) \wedge f] \\
&= \nu'_{f\to v}
\end{aligned}$$

And similarly, for each variable v, the same holds. Thus, $M \leq M' \Rightarrow A(M) \leq A(M')$ In other words, the function A is monotone.

Thus, since the algorithm initially starts off with $\top$, the state of the algorithm comes down with each iteration. This, along with the fact that the lattice is finite, proves that the algorithm is guaranteed to terminate.

From the above discussion, we also get the height of the lattice as a bound on the number of iterations. The height of the lattice is the sum of number of rows in each message. Let there be e edges in the graph; then, there are $2e$ messages being passed in each iteration. Also, let each message be a function of a single variable; then, the height of each lattice is 3. Thus, the maximum number of iterations is $3 \times 2e$.

2.3 Over Approximation

After the message passing algorithm has converged, the projection over a variable v (existential quantification of all other variables) of the conjunction of the functions is estimated by conjoining all the messages coming into v. While computing this "And-Exists" over the functions, it is crucial to ensure over-approximation, otherwise a reachable erroneous state might be overlooked by the model checker.

Theorem 2.3.1 *In every iteration i, for every variable node v and every function node f connected to v, the following two statements hold*

$$\exists_{\overline{v}} F \Rightarrow \mu^i_{v\to f}$$

$$\exists_{\overline{v}} F \Rightarrow \mu^i_{f\to v}$$

where F is the conjunction of all the function nodes.

Proof Notice that this is automatically true for $i = 0$ since all messages are *True*. For an inductive proof, let the hypothesis be $\exists_{\overline{v}} F \Rightarrow \mu^{i-1}_{v\to f}$ and $\exists_{\overline{v}} F \Rightarrow \mu^{i-1}_{f\to v}$ Then,

$$\begin{aligned}
\mu^i_{f\to v} &= \exists_{\overline{v}} \left(\bigwedge_{u\in N_f\setminus v} \mu^{i-1}_{u\to f} \wedge f \right) \\
&\Leftarrow \exists_{\overline{v}} \left(\bigwedge_{u\in N_f\setminus v} \exists_{\overline{u}} F \wedge f \right)
\end{aligned}$$

$$
\begin{aligned}
&\Leftarrow \quad \exists_{\overline{v}} \left(\bigwedge_{u \in N_f \setminus v} F \wedge f \right) \\
&\Leftarrow \quad \exists_{\overline{v}} F
\end{aligned}
$$

And

$$
\begin{aligned}
\mu^i_{v \to f} &= \left(\bigwedge_{g \in N_v \setminus f} \mu^{i-1}_{g \to v} \right) \\
&\Leftarrow \left(\bigwedge_{g \in N_v \setminus f} \exists_{\overline{v}} F \right) \\
&= \exists_{\overline{v}} F
\end{aligned}
$$

■

Corollary 2.3.2 *After the message passing algorithm converges, for all variable nodes v,*

$$\exists_{\overline{u}} F \Rightarrow \bigwedge_{f \in N_v} \mu_{f \to v}$$

Thus, if we start off as every message being true initially, we are guaranteed that the summary over a variable can be over-approximated during any state of the algorithm, the tightest over-approximation of course, being after when the algorithm has converged. It is important to note, however, that the message passing algorithm does indeed produce strict over approximations for some test cases. One very simple example is

$$\exists_{x_2} [\texttt{nxor}(x_1, x_2) \wedge \texttt{nand}(x_1, x_2)]$$

On this graph, the message passing algorithm returns *True* as the answer [Annervaz, 2009], whereas the correct answer is $\overline{x_1}$.

2.4 Comparison with distribution over conjunction

It might be possible that for a given expression, even the best known quantifier schedule (in terms of expected memory requirement) is not tractable. This is often because computing the conjunction of a set of functions before quantifying out a variable results in a large support set. One way of overcoming this problem is to push in the quantifier into the expression to get an over-approximated result. In other words, instead of computing $\exists_x (F \wedge G)$, where F and G are conjunctions of one or more functions, we compute $[\exists_x F] \wedge [\exists_x G]$. The latter has more chances of being tractable because it does not require the explicit computation of $F \wedge G$.

In terms of the factor graph, *distribution over conjunction* can be seen as *splitting a variable node* into different copies. I.e.,

$$\exists_x [F(x) \wedge G(x)] \Rightarrow [\exists_x F(x)] \wedge [\exists_x G(x)] = [\exists_x F(x)] \wedge [\exists_{x'} G(x')] = \exists_{x,x'} [F(x) \wedge G(x')]$$

Thus, distribution over conjunction of x on F and G can be seen as splitting x into x and x', and connecting x to F and x' to G.

However, the result R obtained by running the message passing algorithm on a factor graph G, is always tighter than or equal to the result R' from the graph G', where G' is obtained by splitting a variable node in G. The following two theorems prove the above statement. The first theorem says that the same result can be obtained by splitting the node into two, and adding an *equality function* between

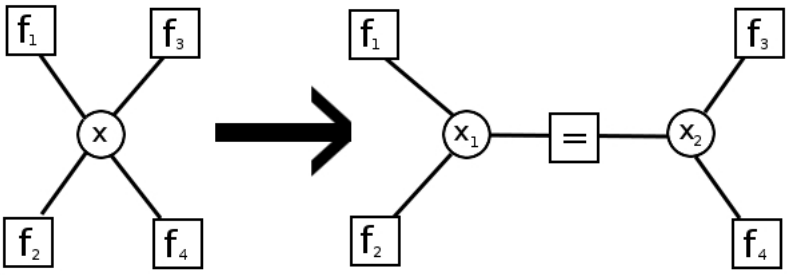

Figure 2.1: Splitting a variable into two and introducing the equality function

the two new nodes. The second theorem says that we can only get a worse (over approximate) result by removing the equality function (and hence getting a subgraph).

Theorem 2.4.1 *Let G be a factor graph. Let G' be obtained by splitting a variable x into x_1 and x_2, and adding an extra function $(x_1 = x_2)$ or more precisely, $(x_1 \wedge x_2) \vee (\neg x_1 \wedge \neg x_2)$ (Fig. 2.1). Then, the result obtained by running the message passing algorithm on G will be the same as that from G'.*

Proof For readability, this proof is provided for the subgraph shown in Fig 2.1, but can be generalized easily. Let M_G^i and $M_{G'}^i$ be the messages passed in the ith iteration while running the message passing algorithm on G and G'. Let M_G and $M_{G'}$ be the messages after convergence. Then we want to prove that $M_G = M_{G'}$. First, we prove $M_G \leq M_{G'}$ Note that $\leq$ concerns only the edges common to both the graphs.

- Base Case : Initially, all messages are true. Thus, $M_G^0 \leq M_{G'}^0$
- Induction Hypothesis : $M_G^k \leq M_{G'}^k$
- Inductive step : Let H and H' be the subgraphs that are not shown in Fig 2.1. I.e.,

$$H = G \backslash \{x \cup N_x\}$$

$$H' = G' \backslash \{x_1 \cup x_2 \cup N_{x_1} \cup N_{x_2}\}$$

Now in the $k+1^{th}$ iteration, the messages coming from H and H' into the subgraphs are easy to handle, since they are computed using the same monotone function. Similar reasoning holds true for the messages from the f_is to all their neighbors. We only need to worry about the message $\mu_{x \to f_i}$ and $\nu_{x_j \to f_i}$ (the messages on the graphs G and G', respectively).

$$\begin{aligned}
\mu_{x \to f_4}^{k+1} &= \mu_{f_1 \to x}^k \wedge \mu_{f_2 \to x}^k \wedge \mu_{f_3 \to x}^k \\
&\leq \nu_{f_1 \to x_1}^k \wedge \nu_{f_2 \to x_1}^k \wedge \nu_{f_3 \to x_2}^k \\
&\leq (\nu_{f_1 \to x_1}^{k-2} \wedge \nu_{f_2 \to x_1}^{k-2}) \wedge \nu_{f_3 \to x_2}^k \\
&= \nu_{x_1 \to =}^{k-1} \wedge \nu_{f_3 \to x_2}^k \\
&= \nu_{= \to x_2}^k \wedge \nu_{f_3 \to x_2}^k \\
&= \nu_{x_2 \to f_4}^{k+1}
\end{aligned}$$

■

Thus, it is established that $M_G \leq M_{G'}$ Now we prove that $M_{G'} \leq M_G$ Note the factor of three with k in the induction hypothesis.

- Base Case : Initially, all messages are true. Thus, $M^0_{G'} \leq M^0_G$
- Induction Hypothesis : $M^{3k}_{G'} \leq M^k_G$
- Inductive step : Once again, the messages coming from H and H' into the subgraphs, and the messages from the f_is to all their neighbors are easy to handle. Essentially, for all these messages,

$$\nu^{3k} \leq \mu^k \Rightarrow \nu^{3k+3} \leq \nu^{3k+1} \leq \mu^{k+1}$$

Now we deal with $\mu_{x \to f_i}$ and $\nu_{x_j \to f_i}$

$$\begin{aligned}
\nu^{3k+3}_{x_2 \to f_4} &= \nu^{3k+2}_{= \to x_2} \wedge \nu^{3k+2}_{f_3 \to x_2} \\
&= \nu^{3k+1}_{x_1 \to =} \wedge \nu^{3k+2}_{f_3 \to x_2} \\
&= \nu^{3k}_{f_1 \to x_1} \wedge \nu^{3k}_{f_2 \to x_1} \wedge \nu^{3k+2}_{f_3 \to x_2} \\
&\leq \nu^{3k}_{f_1 \to x_1} \wedge \nu^{3k}_{f_2 \to x_1} \wedge \nu^{3k}_{f_3 \to x_2} \\
&\leq \mu^k_{f_1 \to x} \wedge \mu^k_{f_2 \to x} \wedge \nu^k_{f_3 \to x} \\
&= \mu^k_{x \to f_4}
\end{aligned}$$

■

For an alternate proof, consider the following lemma :

Theorem 2.4.2 *Let M be a state in a factor graph G. Let M' be obtained by computing only the message $\mu_{n_1 \to n_2}$ from the messages in M, and the rest of the messages being the same. Then, $M_\infty = M'_\infty$, where N_∞ denotes the converged state obtained by running message passing starting from N.*

Proof We know that $M \leq M'$. Let N_k denote the state obtained by running k iterations on a state N. Thus, $\forall_i \;\; M_i \leq M'_i$, and indeed,

$$M_\infty \leq M'_\infty$$

However, also notice that $M' \leq M_1$. Thus, $M'_i \leq (M_1)_i = M_{i+1}$, which gives

$$M'_\infty \leq M_\infty$$

■

Corollary 2.4.3 *The state obtained by running the message passing algorithm can also be obtained by updating messages in any order until convergence.*

We now present an alternative proof to theorem 2.4.1

Proof Normally, the message passing algorithm computes all messages in an iteration. But we have seen that updating single messages in between iterations does not affect the final converged state. Let the function representing an iteration for the original graph G be A. Consider the following alternative iteration A' for G':

1. Update $\nu_{x_1 \to =}$ and $\nu_{x_2 \to =}$
2. Update $\nu_{= \to x_1}$ and $\nu_{= \to x_2}$

3. Update all messages except those between x_1, x_2 and $=$.

Now we will prove that A' is the same as A (for the corresponding edges). For the edges that do not concern x, there is no difference. Also, for the edges incoming into x, A is the same as A'. Thus, we just need to prove that the message going out of x in G, say $\mu_{x\to f_4}$ and the corresponding message in G', $\nu_{x_2\to f_4}$ are equal (up to substitution of x with x_2). This turns out to be straightforward:

1. $\nu^{k+1}_{x_1\to=} = \nu^k_{f_1\to x_1} \wedge \nu^k_{f_2\to x_1}$
2. $\nu^{k+1}_{=\to x_1} = (\nu^k_{f_1\to x_1} \wedge \nu^k_{f_2\to x_1})_{x_1->x_2}$ (where $(f)_{a->b}$ means substituting a with b in f).
3. $\nu^{k+1}_{x_2\to f_4} = (\nu^k_{f_1\to x_1} \wedge \nu^k_{f_2\to x_1})_{x_1->x_2} \wedge \nu^k_{f_3\to x_2}$

Thus, $\nu^{k+1}_{x_2\to f_4}$ is the same as $\mu^{k+1}_{x\to f_4} = \mu^k_{f_1\to x} \wedge \mu^k_{f_2\to x} \wedge \mu^k_{f_3\to x}$. Now since A' gives the same convergence as running simple factor graph iterations on G', we can conclude that the convergence on G and G' are the same.

■

Theorem 2.4.4 *Suppose a factor graph G is a subgraph of another factor graph H. Let M_G and M_H be the tuple of converged messages (after running message passing algorithm) on the edges of G and on the corresponding edges of H, respectively. Then,*

$$M_G \geq M_H$$

where $\geq$ relates only the edges in G, to the corresponding edges in H.

Proof To complete this proof, we use the monotonicity of the function that computes the messages during an iteration. To use this property, we consider two extended message tuples N_H and N_G.

N_H is the set of messages on all edges of H. N_G is created to be of the same size as N_H. For the edges that are contained in the subgraph G, N_G is the same as M_G, but for other edges, N_G is *True*.

Furthermore, let N^i_G and N^i_H be the messages during the ith iteration of message passing algorithm on graphs G and H respectively. Now we induct on i to prove that $N^i_G \geq N^i_H$ for all i.

- Base case : Initially, all messages are true. Thus, $N^0_G \geq N^0_H$
- Induction hypothesis : $N^k_G \geq N^k_H$
- Inductive step : Let the functions used to compute M^{k+1}_G and M^{k+1}_H be $g(M^k_G)$ and $h(N^k_H)$. Notice that because all messages that are in N_G but not in M_G are *True* and G is a subgraph of H,

  $$g(M^k_G) = h(N^k_G)$$

  Now we use the monotonicity property of h to say that

  $$N^k_G \geq N^k_H \Rightarrow M^{k+1}_G \geq M^{k+1}_H$$

  This fact, in conjunction with the fact that all messages that are in N^{k+1}_G but not in M^{k+1}_G are defined to be *True*, gives us

  $$N^{k+1}_G \geq N^{k+1}_H$$

Thus, after convergence, $N_G \geq N_H$ and hence,

$$M_G \geq M_H$$

■

If a distribution over conjunction, by dint of node splitting, leads to breaking of all cycles in the factor graph, then the result from running the message passing algorithm on the factor graph would be exactly equal to the result of distribution of the quantifier over conjunction. We have also proved above that the original factor graph can only give tighter results than any factor graph derived by splitting of variables. Thus,

Theorem 2.4.5 *Let $P = (F, V)$ be an instance of the and-exists problem, for which the factor graph is G. Let P' be a way of distributing some of the quantifiers over conjunction, for which the graph is G'. Then, the result R, obtained by running the message passing algorithm over G, is tighter than the result R' obtained by running the message passing algorithm over G'. I.e.,*

$$R \Rightarrow R'$$

Corollary 2.4.6 *Let $P = (F, V)$ be an instance of the and-exists problem, for which the factor graph is G. Let P' be a way of distributing some of the quantifiers over conjunction, for which the graph G' is acyclic. Then, the result R obtained by running the message passing algorithm over G is tighter than the exact answer R' of P'.*

This is a generalization of the result proved in Annervaz [2009] which states that the original factor graph gives tighter results than the distribution over conjunction in which all quantifiers are distributed over all conjunctions.

2.5 Projection over multiple variables

The message passing algorithm that has been discussed so far can only compute the projection over the variable or set of variables in a single node. Thus, in order to compute the projection over a particular set, one way would be to merge all the variables of that set in a single node.

Merging variable nodes A factor graph $G' = (V', E')$ is said to be the result of merging variable nodes x and y in graph $G = (V, E)$ if

- V' has the combined node $w = (x, y)$ in place of x and y:

$$V' = (V \setminus \{x, y\}) \cup \{w\}$$

- E' consists of all the edges in E, with any edge to x or y replaced with an edge to w :

$$E' = \{(v, f) \in E | v \neq x \wedge v \neq y\} \cup \{(w, f) | (f, x) \in E \vee (f, y) \in E\}$$

The node w is said to be the *merged node* of x and y.

However, the space complexity of messages coming in and out of a node is worst case exponential in the size of the support set of the node. Also, the (very loose) upper bound on the number of number of iterations that is shown in section 2.2 is actually exponential in the size of the support set for graphs with merged nodes. Thus, too many variables in a merged node are to be avoided.

A way around this would be to compute the projections on subsets of the required set, and conjoin these projections. In other words, instead of computing $\exists_{\overline{X \cup Y}} F$ we compute the conjunction $\exists_{\overline{X}} F \wedge \exists_{\overline{Y}} F$. This conjunction is an over approximation, since both the expressions conjoined are over approximations.

Thus, if keeping $X \cup Y$ as a single node causes the algorithm to blow up, an over approximation may be computed instead. For the rest of this section, this method is referred to as the *conjunction of individual projections*.

To complete the discussion on merging and conjunction methods, we must compare the accuracy of the results obtained by these two methods. Annervaz [2009, Theorem 7.1] states that the result obtained by running the message passing algorithm on a graph with two variables merged is tighter than the result obtained by running the algorithm on the original graph and conjoining the individual projections. However, while restating the proof more formally, a loophole was found. As of now the statement is still a conjecture.

2.6 Leveraging quantifier scheduling

As explained before, given a conjunction of a set of formulae and a variable to be existentially quantified out, exact computation can be done only by computing the conjunction of a certain subset of functions. For example,

$$\exists x[f(x,y) \wedge g(y,z) \wedge h(x,z) \wedge w(z)] \leftrightarrow \exists x[f(x,y) \wedge h(x,z)] \wedge g(y,z) \wedge w(z)$$

This interestingly leads to different choices when multiple variables are to be eliminated. For example, in the following formula, we can either eliminate y first, followed by x, or x first, followed by y.

$$\exists x[\exists y[f(x,y) \wedge g(x,y) \wedge h(y,z)]] \tag{2.3}$$

$$\leftrightarrow \quad \exists y[h(y,z) \wedge \exists x[f(x,y) \wedge g(x,y)]] \tag{2.4}$$

In general, different orderings of the variables lead to different computation trees. This ordering may be very crucial in terms of tractability. The main bottle neck for such operations is usually the peak memory requirement. The order that gives the smallest peak memory requirement depends heavily on the representation (BDD, AIG-graphs, etc) used for the logic. For BDDs, finding the best possible variable ordering is NP complete [Bollig and Wegener, 1996]. Instead, a fairly good characterization of the peak memory requirement is the size of the largest support set of any function represented.

Largest Support Set (LSS) The LSS of a quantifier schedule Q is the size of the largest support set among all functions that are stored explicitly while computing Q.

Notice that LSS of (2.4) is 2, whereas the LSS for (2.3) is 3. In general, finding the best ordering is difficult, and instead heuristics have been discovered.

If for a given expression, even the best ordering also cannot bring down the LSS sufficiently, then the result can be over approximated *distribution over conjunction*, i.e., by distributing the quantification over the conjunction. However, as seen in theorem 2.4.5, after a certain point, distribution over conjunction gives worse results than the message passing algorithm. Hence we investigate methods that avoid distribution over conjunction as much as possible.

In terms of factor graphs, eliminating a certain variable can be seen as collapsing the variable and its neighboring function nodes into a new function node (Fig 2.2). For example, in the factor graph of expression (2.4), removing x results in collapsing of x, f and g into a new function node representing $\exists x[f(x,y) \wedge g(x,y)]$.

This way, given a factor graph, variables can be eliminated by successive collapsing of variable nodes. If we happen to have a good variable ordering that allows us to compute the expression exactly, the factor graph will also be able to compute the result accurately, without having to resort to message

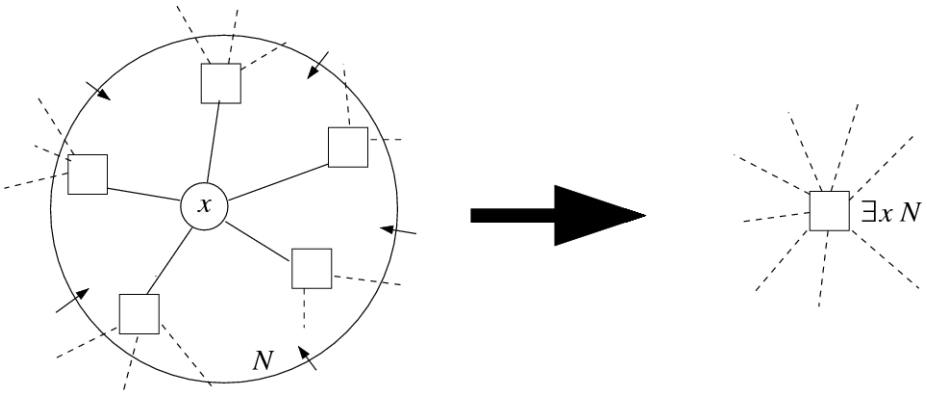

Figure 2.2: Elimination of x from a factor graph

passing. It might, however, also be the case that no quantifier schedule is efficient enough to do an exact computation.

In such a case, either the rest of the quantifier schedule can be dropped and the message passing algorithm can be used to compute the final projection, or, the message passing algorithm can be used to compute just the current elimination, and the result could be used to continue with the schedule.

While the comparison of these two methods has not been done so far, we will now present some observations about the second method, i.e., to use the message passing algorithm to compute $\exists_x N$, where N is the conjunction of the neighboring functions of x.

The first observation is that, instead of using just the factor graph of N, we can use the entire factor graph. By theorem 2.4.4, we know that this would give a tighter result than that given by using just N. Also, let N' be the functions that are not neighbors of x and let V be the variables other than x. Then, let

$$\begin{aligned}
\exists_x[N'(V) \wedge N(V,x)] &= \exists_x[N'(V) \wedge N(V,x) \wedge N'(V)] \\
&= [N'(V) \wedge [\exists_x N(V,x) \wedge N'(V)]] \\
&\Rightarrow [N'(V) \wedge \texttt{SPA}[\exists_x N(V,x) \wedge N'(V)]]
\end{aligned}$$

In short, the result obtained from using the entire subgraph for computing $\exists_x N$ is an under approximation of that obtained by just the factor graph of N, but using it would still lead to an over approximation of the final result that is to be calculated.

Also, recall that the final result of the message passing algorithm is obtained by conjoining the messages coming into the appropriate variable nodes. Since we are adding this result into the original factor graph, we might as well create separate function nodes for each message. (Since a factor graph represents the conjunction of all the function nodes.) More functions would lead to smaller support sets per function, which would very likely lead to lesser peak memory requirement. At the same time, this would lead to lesser number of cycles, and hence, a more accurate result.

2.7 Reusing of messages

As evident in later chapters, we might often require convergence in almost similar factor graphs. For example, in order to make a graph acyclic (section 3.3), if we assign a variable to both true and false to get two factor graphs, the factor graphs will be similar except for the variable node, its neighboring function

nodes, and their immediate neighbors. Thus, it would make sense to explore how to reuse messages from a similar factor graph to reduce computation time, and get faster convergence. In order to approach in this direction, we prove the following theorem :

Theorem 2.7.1 *Let G be a factor graph on which we start the message passing algorithm with the message tuple $M_0 = \top$, and let M_i be the message from the ith iteration. Let N be a message tuple created by mixing corresponding messages from M_i, where $i \in [0, k]$ for some k. I.e., each edge is given a message from a corresponding message in any of the iterations. Then, convergence on starting from N would lead to the same messages as that from M_0, i.e., $N_\infty = M_\infty$*

Proof By construction of N, and the fact that messages in M only get tighter on successive iterations, we know that $M_0 \leq N \leq M_k$. Since an iteration is a monotone function, we get $M_i \leq N_i \leq M_{k+i}$, and in particular, $M_\infty \leq N_\infty \leq M_\infty$. Thus proved.

Thus, we now know how to use the pre-computed messages of a factor graph G in G itself. For comparing the messages of G and a modified graph G', we use the fact that till iteration k, edges in G' which are at a distance more than k from the changed part of the graph must have the same messages as G.

Distance of an edge from a set of nodes An edge e which is connecting nodes n_1 and n_2 is said to be at distance 1 from n_1 (and n_2). An edge e_1 which has a node common with edge e_2 is said to be adjacent to e_2. An edge e_1 which is adjacent to an edge e_2 at a distance d from node n (and no other edge adjacent to e_1 is at a closer distance), is said to be at distance $d + 1$ from n. The distance of an edge e from a set of nodes N is defined as the distance of the closest node in N from e.

Theorem 2.7.2 *Suppose G and G' are factor graphs which differ only in N and N', i.e., $G' = (G \setminus N) \cup N'$ and $G = (G' \setminus N') \cup N$. Also, let $M, M_1, ...M_k$ be the message tuples in G in successive iterations of the message passing algorithm, and similarly $M'(= M), M'_1, M'_2, ...M'_k$ be for G'. Then, for an edge e common to both the graphs and at a distance at least d from N and N', the messages will not differ in the two graphs till the dth iteration of the message passing algorithm. I.e.,*

$$\forall\ 0 \leq i < d \quad M_i(e) = M'_i(e)$$

Proof We prove this by induction on i, the number of iterations. The base case $i = 0$ is simple, since by definition $M_0 = M'_0$.

Now for the induction hypothesis, we assume that after iteration i, edges which are at a distance $i+1$ or more have similar messages.

Hence, in the $i + 1^{th}$ iteration, the computation of nodes at a distance $i + 2$ or more are going to have the same messages, as the incoming edge messages must be same (since by definition, their distance $\geq i+1$) and also the structure of the graph is the same (the difference in the graph is at least $i+1$ edges away). Thus proved.

From this theorem, we get the fact that for graphs G and G' which differ in N and N', messages on a common node e, which is at a distance at least d from N and N', will be the same until iteration $d-1$. Thus, if message passing in G has converged in k iterations, the converged messages can be reused in all edges at a distance $(k+1)$ or more from the difference in the graphs. This opens up a large potential for reuse of messages, but in order to put it into practice we need one more piece of the puzzle.

Suppose we converge on graph G in k iterations, and in graph G', we use the messages of graph G at appropriate positions, and graph G' now converges in l iterations. Then, what can be said about k',

i.e., the number of iterations that G' would have taken had it started from $M'_0 = \top$? (k' is essential for reusing the converged messages of G' for future graphs.) All we can say for certain is that $k' \leq l + k$, but this is not sufficient since this would imply that k must necessarily increase over successive uses (whereas experimentally it is almost always 3 and occasionally 4). Intuition suggests that $k' \leq max(l, k)$, but as of now it is still a conjecture.

An observation in the above discussion is that if we were to put message μ_e from the kth iteration on some edge e and $\top$ in neighboring edges, then in the next iteration, the newly computed message might be looser than the old one, i.e., $\mu'_e \geq \mu_e$. Convergence is still guaranteed, but we must ask ourselves whether we should keep the old tight message in search of faster convergence, or will doing that lead to under approximate results? It turns out that keeping the tighter message will still lead to over approximations, and the proof can be easily obtained on lines similar to Theorem 2.7.1, but there is also an alternative proof.

Suppose we define the message update rule in message passing to take the tighter of the old message and the newly computed message. Then, an iteration is still a monotonic function (since $m_1 \leq n_1$ *and* $m_2 \leq n_2 \Rightarrow min(m_1, m_2) \leq min(n_1, n_2)$), performing message passing on an acyclic graph still gives the correct answer, and on a cyclic graph still gives over approximate results. As a result, all results proved so far would continue to hold on such an algorithm.

Chapter 3

Factor Graph Implementation

3.1 Factor Graph

Keeping in mind the need for low-level performance tweaks in order to build a competitive tool, and also the smooth compatibility with Cudd, we decided to write an implementation in C. `factor_graph`, `fgnode`, `fgedge`, `fgedge_list` and `fgnode_list` are the various structs which are described below. A short two-line summary is as follows : The graph exists as a set of function nodes and variable nodes, which have lists of edges to define their neighbors. Each edge has a link to connected nodes, as well as the messages being passed to and fro.

3.2 Structs

A major design choice while implementing a graph structure is whether to keep the connectivity information as an adjacency list or an adjacency matrix. In our case, we need to work with all the messages passing to and fro between nodes, so it makes sense to keep an edge as a separate tangible structure rather than something implicit. `fgedge` is a struct that contains links to a variable node and a function node (both `fgnode`s) and BDD pointers to the messages being passed to and fro. Other members such as the birth and death time-stamps, edge id, and message convergence booleans will be explained as we go along.

Each edge is kept in multiple lists - in the neighbor lists of the variable as well as function nodes, and in a list of all edges in the `factor_graph`. Hence, we created a separate linked list data structure to store edges, rather than making `fgedge` a list by putting `prev` and `next` pointers.

An `fgnode` can be either a variable node or a function node depending on the value of `type` field. Besides `type`, there is of course the `fgedge_list * neigh`, which is the list of neighboring edges, and `bdd_ptr *f` and `bdd_ptr *ss`. For a variable node, both `f` and `ss` have one element each, which is a BDD cube of the variables in the variable node. For a function node, `f` contains the functions that are a part of the function node, and `ss` contains their support sets. The need for multiple functions in a node comes when we merge function nodes while trying to render the graph acyclic. `parent` and `color` are members which are used while doing a breadth first traversal. Although what they are may be clear, why they are needed shall be explained in section `factor_graph_make_acyclic`.

`fgnode_list` is a simple data structure to store `fgnode`s in a list. It is used, for example, to keep lists of all function nodes and variable nodes in a graph, and a queue of nodes that are to be explored while doing a BFS.

3.3 Making the graph acyclic

There are essentially two ways in order to make a graph acyclic. One way is to merge nodes. Theoretically, two function nodes can be merged into a single node by conjoining the two functions. Similarly, two variables can be merged into a single node by taking their cross product. In practice, function nodes are merged into one by concatenating the arrays of the function nodes, as an array implicitly represents the conjunction of its functions. Variable nodes are also not practically merged by creating a new variable that represents the cross product, but by keeping both the variables and representing the cross product implicitly. For example, while computing the message to be passed, quantifying out both variables individually is equivalent to quantifying out the (hypothetical) merged variable

The other way to make a graph acyclic is to eliminate variable nodes by setting them to true and false, and combining the results of the two *subproblems*. If a variable is to be universally (existentially) quantified out, the two results need to be conjoined (disjoined). If the variable x is to be kept intact, the results f_0 and f_1 can be combined as $(x \vee f_0) \wedge (\neg x \vee f_1)$. The complications in case of quantified boolean formulae (where such quantifiers are intermixed) has lead to the SRT data structure, which is discussed in the next section.

Each time a variable is used to "split" the problem into two sub problems, we are losing out on the sub-exponential computation by BDDs. If we were to split the problem recursively by all variables, it would amount to a brute force exponential computation of the result. While it is true that BDD-computations are also exponential in the worst case, they do work much better in practice. Clearly, too many variable splits will loose out on this "usual case" performance.

On the other hand, BDDs are known to choke out of memory, especially during quantification operations. Even though the beginning and starting sets of BDDs are manageable, the intermediate computation leads to a blow-up in memory, often causing it to thrash (continuous page in of virtual memory), or even simply running out of memory. Although intermediate BDD sizes are completely unpredictable (from all practical perspectives), a rough estimate is the size of the support set (although the support set does indeed limit the worst case size of a BDD, BDD sizes are so unpredictable that it is worth emphasizing that support set sizes are nothing but a crude but computationally cheap and conceptually simple indicator). As a result, we can see how aggressive merging of nodes may lead to a memory blowup. Indeed, merging all nodes together is the same as conjoining all the function nodes together and quantifying out all the variable nodes, with quantifier scheduling being perhaps the only degree of freedom.

In short, splitting the problems versus merging of nodes is (roughly) a tradeoff between peak memory and overall time taken, and the function `factor_graph_make_acyclic` is conceptualized with this in mind.

`factor_graph_make_acyclic` does a breadth-first-traversal of the factor graph, and merges nodes whenever it finds a cycle. It also accepts a threshold parameter λ, such that it will merge nodes together only if the resulting BDDs have a support set smaller than λ. When it cannot remove a cycle without exceeding λ, it returns a variable from the cycle, as a recommendation for a split.

Thus, this function can be called repeatedly with some value of λ. If it does not succeed, the variable it returns can be used to split the problem and make the graph acyclic. The value of λ thus acts like a tuner between memory and time. A high λ will lead to larger BDDs, and a low λ will lead to more splits.

3.4 Checkpoints

A variable can be assigned a value in a factor graph by simply assigning the variable in all its neighboring function nodes and removing the variable node from the graph. However, a split requires us to assign two values, both true and false. Creating copies of the factor graph is memory inefficient, since at one point of time we are only concerned with one copy of the factor graph. At the same time, it is impossible to obtain the factor graph with $x = \bot$ from the graph with $x = \top$. Instead, we give the facility of *checkpoints* in the factor graphs. Essentially, a checkpoint can be created before setting x to $\bot$. Then after we are done with the computation of $x = \bot$, we can roll back to the checkpoint and then again set x to $\top$ (maybe after setting a checkpoint again).

This notion of checkpoints is implemented by adding the concept of time. The `factor_graph` struct has an integer member called `time`, which represents the current time. Also, each node has `born` and `died` time stamps. When a new `fgnode/fgedge/fgnode_list/fgedge_list` is created, its `born` is set to `time`, and its `died` is set to infinity. When the node is to be deleted, its `died` is set equal to `time`. Hence, while doing any processing with the factor graph such as message passing or `make_acyclic`, elements with `died` $\leq$ `time` are treated as non-existent. A checkpoint is set by simply doing `fg->time++`. Garbage collection is done when calling `factor_graph_rollback`, in which time is decreased by one, and all elements with `born` $\geq$ `time` are freed from memory. An example for using checkpoints is given below :

```
fg->time++;
factor_graph_assign_var(fg, v);
<some processing>;
factor_graph_rollback();
fg->time++;
factor_graph_assign_var(fg, not_v);
<some processing>;
factor_graph_rollback();
```

3.5 Convergence

Message passing is initiated by setting each message in each edge to be true. Then, we apply the update rule to each node until no more changes happen in the graph. The update rule sets the outgoing message from a node to a neighbor, depending on the incoming messages from the other neighbors. For a variable node, the outgoing message to a neighbor is simply the conjunction of all the messages incoming from all other neighbors. For a function node F, the outgoing message to a neighboring variable x is the result of quantifying out all variables other than x from the conjunction of all incoming messages from variables other than x and the function(s) in the function node.

This message passing is done as a traversal. Whenever a message is updated, the destination node gets added to the TODO-list. All nodes from the TODO-list are processed until the list becomes empty. Duplicates are avoided from entering the list with the use of the `color` boolean.

The convergence of the message passing algorithm is guaranteed (section 2.2), but the theoretical upper bound on the number of iterations is quite bad compared to experimental results. Per node, we need to compute the outgoing message for each node, using the incoming messages from all other nodes. This way to look at it gives a $\mathcal{O}(n^2)$ algorithm, where n is the number of neighbors. A dynamic programming algorithm allows us to compute all out going messages in $\mathcal{O}(n)$ BDD calls. Essentially,

we compute all the incremental conjunctions in a sequence, and also the incremental conjunctions in the reverse sequence (see algorithm given below). Then the conjunction of all messages except one is a single conjunction of one of the BDDs in the first sequence and another in the second sequence. Also, since there is already an in built cache in Cudd, this can be achieved by just conjoining nodes in the appropriate order. A pseudo code is as follows:

```
void update_messages(fgnode *v) {
    Let N = array of messages coming into v;
    Let n = N.size() - 1; Let B and Brev = two BDD arrays;
    B[0] = N[0];
    Brev[n] = N[n];
    for i = 1 to n do
        B[i] = B[i − 1] ∧ N[i];
        Brev[n − i] = Brev[n − i − 1] ∧ N[n-i];
    end for
    for i = 0 to n do
        BDD l = (i > 0 ? B[i − 1] : ⊤);
        BDD r = (i < n ? Brev[i + 1] : ⊤);
        v.outgoing[i] = l ∧ r; //v.outgoing is the array of outgoing messages
    end for
}
```

Chapter 4

Split Record Table

Split Record Table, or SRT for short is essentially a data structure for managing splits in factor graphs while applying the and-exists operator on quantified boolean formulae. Suppose we have a QBF $\exists x_1 \forall x_2 \exists x_3\ \phi$. Hence, paying due regard to the quantifier ordering, we start with quantifying x_3 out of ϕ. Suppose while quantifying x_3, we need to split on the variable x_2. In other words, we substitute true and false into x_2 and end up with two solutions $\exists x_3\ \phi_{x_2=\top}$ and $\exists x_3\ \phi_{x_2=\bot}$, which we call $\psi_\top$ and $\psi_\bot$. Thus, since x_2 was a universally quantified variable, to get the final answer we simply conjoin these two answers and quantify out x_1. I.e., $\exists x_1 \forall x_2 \exists x_3\ \phi \equiv \exists x_1\ (\psi_1 \wedge \psi_2)$

On the other hand, if while quantifying out x_2, we end up with $\exists x_3\ \phi_{x_1=\top}$ and $\exists x_3\ \phi_{x_1=\bot}$, which we call $\xi_\top$ and $\xi_\bot$. Now the original problem involves not just the disjunction of these ξ, but something slightly more complicated.

$$\exists x_1 \forall x_2 \exists x_3\ \phi \equiv \exists x_1 \forall x_2\ (x_1 \vee \xi_\bot) \wedge (\sim x_1 \vee \xi_\top) \tag{4.1}$$

$$\equiv \exists x_1\ [(x_1 \vee (\forall x_2\ \xi_\bot)) \wedge (\sim x_1 \vee (\forall x_2\ \xi_\top))] \tag{4.2}$$

Equation 4.1 means that the removed variable must be reintroduced into the results immediately after they are computed. However, equation 4.2 tells us that we may apply $\forall x_2$ into the sub-results and then combine them afterwards. Although this seems promising, splits over multiple variables leads to complication in the aggregation of sub results. The Split Record Table or the SRT for short is a data structure for storing results of subproblems in an organized fashion so that aggregations can be performed correctly.

4.1 SRT definitions

Split Record Table An SRT for a QBF problem $\mathcal{Q}Y\ \phi$ is a binary tree in which each non-leaf node represents the assignment of a variable with true and false. Each leaf stores $\phi_{\mathcal{A}}$ (i.e., ϕ substituted with $\mathcal{A}$) where $\mathcal{A}$ represents all the assignments from the root node to that leaf node.

For the purpose of this report, we call the branch with assignment false as the left branch and the one with true as the right branch.

We now describe how to obtain the answer to the QBF using already existing splits. Consider a generalized QBF $\mathcal{Q}Y\ \phi$, where $\mathcal{Q}$ are the quantifier types, Y is the set of variables which are quantified, and X is the set of variables which are not, and ϕ is a boolean formula in X and Y. Thus, the result of this quantification is a result of X. For a split on $x \in X$, we get

$$\mathcal{Q}Y\ \phi \equiv \mathcal{Q}Y\ ((x \vee \phi_{x=\bot}) \wedge (\neg x \vee \phi_{x=\top})) \tag{4.3}$$

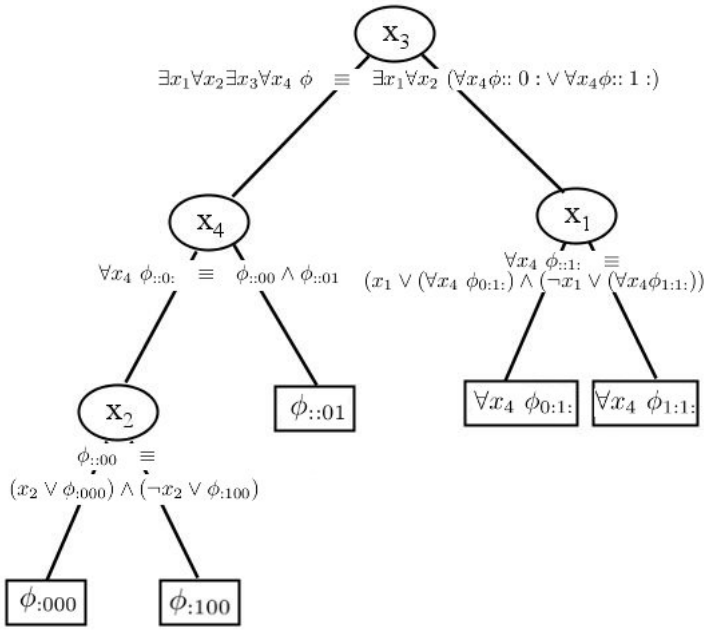

Figure 4.1: A sample Split Record Table

$$\equiv \quad (x \vee (\mathcal{Q}Y\ \phi_{x=\bot})) \wedge (\neg x \vee (\mathcal{Q}Y\ \phi_{x=\top})) \tag{4.4}$$

$$(\because [\mathcal{Q}Y\ \phi]_{x=v} \equiv \mathcal{Q}Y\ [\phi_{x=v}]) \tag{4.5}$$

where equation 4.4 is said to be *the pack operator* applied on subproblems $\mathcal{Q}Y\ \phi_{x=\top}$ and $\mathcal{Q}Y\ \phi_{x=\bot}$.

Similarly, for a split on $y \in Y$, we have

$$\mathcal{Q}Y\ \phi \quad \equiv \quad \mathcal{Q}Y_L\ (\mathcal{Q}Y_R\ \phi_{y=\bot}\ \texttt{op}\ \mathcal{Q}Y_R\ \phi_{y=\top}) \tag{4.6}$$

where $\texttt{op}$ is $\wedge$ or $\vee$ depending on whether the quantifier of y is universal or existential, and $Y = Y_L\ y\ Y_R$. Accordingly, for these two cases, we define the *pack* and the *quant* operator.

Pack operator The pack operator on children $F_\bot$ and $F_\top$ of a node with split variable x is defined as $Pack(x, F_\bot, F_\top) = (x \vee F_\bot) \wedge (\neg x \vee F_\top)$.

Quant operator The quant operator on children $F_\bot$ and $F_\top$ of a node with split variable x is defined as $Quant(x, F_\bot, F_\top) = F_\bot\ \texttt{op}\ F_\top$ where $\texttt{op}$ is either $\wedge$ or $\vee$ depending on whether the quantifier of x is universal or existential.

Thus, since the operator that is to be applied to a node depends on what is the set of quantifiers that has remained, which in turn depends on the splits that have occurred previously, we will need the notion of **innermost ancestor** or a node.

Definition The ancestor split set or for short, the **ancset** of a node, as is the set of all variables which have been split upon as ancestors to a node. A node itself, is NOT a part of its ancset. The ancset of a root node is empty. Furthermore, the variable x_i which is the innermost variable in the ancset of a node N is called the **innermost ancestor** of N. The innermost ancestor of the root node is a hypothetical artificial variable, added to the problem, outside of all other variables.

Definition Suppose we have a non-leaf node with split variable x_k and innermost ancestor x_t. Then, if x_k is inside of x_t then the node is called a **quant node**, and if x_t is inside of x_k then the node is called a **pack node**.

For the sake of convenience, we assume to the set of variables to be $\{x_1...x_n\}$ and that if $i < j$ then the variable x_j is inner to the variable x_i in the quantifier ordering $\mathcal{Q}$. We take x_0 to be an artificial variable not part of the original problem.

Node Subproblem For a node N with innermost ancestor x_t, split variable x_k and the set of assignments $\mathcal{A}$, we define the node subproblem to be $\mathcal{Q}_{t+1}x_{t+1}\mathcal{Q}_{t+2}x_{t+2}\ldots\mathcal{Q}_n x_n\ \phi_{\mathcal{A}}$.

Node Operation For a leaf node N with innermost ancestor x_t and the set of assignments $\mathcal{A}$, the operation is defined as $\mathcal{Q}_{t+1}x_{t+1}\mathcal{Q}_{t+2}x_{t+2}\ldots\mathcal{Q}_n x_n\ \phi_{\mathcal{A}}$. For a non-leaf node N with split variable x_k, innermost ancestor x_t and child outputs ϕ_0 and ϕ_1, the output is $\mathcal{Q}_{t+1}x_{t+1}\mathcal{Q}_{t+2}x_{t+2}\ldots\mathcal{Q}_{k-1}x_{k-1}\ Op(x_k, \phi_0, \phi_1)$. If N is a quant node, i.e., if $k > t$, then $Op(x, f, g)$ is the quant operator on f and g using the variable x, and if N is a pack node, i.e., if $t > k$, Op is the pack operator.

Theorem 4.1.1 (Correctness of Node Operations) *If we apply the operations of a node to the solutions of its children, then we get the solution of the node.*

Proof Proof for leaf nodes is trivial. For non-leaf pack nodes, $k < t$. Thus, the innermost ancestor of its children is also x_t, and the only difference between its own output and its children's outputs is the assignment of x_k to true and false in the children. Hence we can get the output as the Shannon expansion of its cofactors, which is nothing but the pack operator. For a non-leaf quant node N, $k > t$. This means that the innermost ancestor for its children is nothing but x_k, and now not only do the children have an extra assignment of x_k, but N also has an extra quantifiers on variables x_{t+1} to x_k. Since the assignments in the children are used to compute the quantifier on x_k is handled by the quant operator, and the other quantifiers are computed explicitly, the proof holds for non-leaf quant nodes as well.

■

As an example, consider the SRT in figure 4.1 for the problem $\exists x_1 \forall x_2 \exists x_3 \forall x_4\ \phi$. The figure shows the subproblem of a node, and also how it can be obtained from the child subproblems (for interior nodes).

Thus the SRT is a data structure that stores the various assignment splits in a tree like fashion. This section has also described the various operators that are to be given to the nodes so that applying these operators bottom up gives us the final solution. We now proceed to give an algorithm to use this structure along with factor graphs to solve the QBF.

4.2 QBF_solve

A rough very high level view of the `QBF_solve` algorithm is given in figure 4.2. Each iteration of the algorithm takes the innermost variable and computes it explicitly. In other words, if the SRT belongs to a problem $\mathcal{Q}X\mathcal{Q}_i x_i\ \phi$ before some iteration, then after the iteration the SRT is now an SRT of the problem $\mathcal{Q}X\ \psi$ where $\psi = \mathcal{Q}_i x_i\ \phi$ has been computed explicitly. For leaf nodes which do not have x_i in their ancestor assignments, $\psi_{\mathcal{A}}$ is computed explicitly. For leaf nodes which have a split on x_i, we instead perform a `merge` operation on the x_i split ancestor itself. The details of the merge operation are in section 4.3.

Various optimizations can be applied to the above algorithm :

- When we quantify out a variable from a conjunction of functions, many of the functions which do not depend on the variable will remain the same. As a result, most of the leaves of the SRT might have many conjoins common. Thus, it is better to keep a global set of conjoins, which are implicitly

```
create an SRT with one node containing the CNF;
while v = innermost_variable() do
   for each node n split on v do
      n.merge();
   end for
   for each leaf node n do
      Q = (v is universal ?  ∀ :  ∃);
      n = Qv n;
   end for
   remove v from the quantifier list;
end while
```

Figure 4.2: A brief view of the `QBF_solve` algorithm

conjoined to each leaf. To implement this, we start off with the root node being a leaf with no conjoins, and all the functions in the global array. Then, as variables are quantified out, clauses depending on the variables are taken out, and appended to each leaf before the quantification.

- For universally quantified variables, we need not append these conjoins to each leaf. Instead, just quantifying out the variable from the global array itself will do.
- For existentially quantified variables also, if none of the leaves contain the variable, we may simply quantify the variable from the global array, and need not append anything to the leaves.
- There are many deductions which are possible while seeing the structure of a QBF. In fact such optimization tricks are the main driving force behind most current state of the art QBF solvers. Section 4.4 give a brief description on how we plan to do such deductions on BDDs rather than in CNF.

A pseudo code containing all these optimizations is given below for completeness.

```
QBF_solve(filename FN)
{
   SRT T = parse_qdimacs_srt(FN);
   //parses the set of clauses and the set of quantifiers
   quant Q = T.vars();
   CNF_vector C = T.global_clauses();
   BDD* VarDep;
   while (not Q.empty()) do
      vars V = Q.innermost();
      //the innermost quantified variables
      for each variable v in V do
         T.merge(v);
         //merges each split node on v, see section 4.3
      end for
      if ( Q.is_universal(V) ) then
         for each clause c in C do
            c.drop_vars(V);
```

```
        //notice how the clauses in C are not appended to leaf nodes
    end for
    if ( VarDep contains some variable of V ) then
        // whether clauses of existential variables quantified out
        // till now depends on any variable from V or not
        // safe version of whether any leaf is dependent on V
        for each leaf node L in T do
            L.forall(V);
            //convert each func in L into CNF, drop V, convert back;
            T.deduce(L);
            //may lead to further splits, and L might no longer be
            //a leaf node. The leaf node iterator has to be able to
            //handle this
        end for
    end if

else if ( Q.is_existential(V) ) do
    CV = C.filter(V);
    //filters clauses from C that depend on V
    //and removes them from C
    if ( VarDep contains some variable of V ) then
        for each leaf node L in T do
            L.append(CV);
            //apply the split history of L to CV, add these clauses to L
            fg = make_factor_graph(L);
            //create a new factor graph from L
            factor_graph_eliminate(fg, V, L);
            //recursive function, may cause splits in L if cyclic
            factor_graph_delete(fg);
            T.deduce(L);
            //similar to deduce after removal of universal variables
        end for
    else
        // no leaves depend on V
        Lnew = T.make_leaf(CV);
        // no leaf depends on V, form a global leaf from CV
        fg = make_factor_graph(Lnew);
        factor_graph_eliminate(fg, V, Lnew);
        factor_graph_delete(fg);
        for each leaf node L in T do
            L.append(Lnew);
            T.deduce(L);
        end for
    end if
    VarDep = VarDep ∪ (CV.support_set() \ V);
```

```
        // mark variables coming from CV into T
      end if
      Q.remove_innermost();
      // removes innermost block
    end while
}
```

4.3 Merging of nodes

Suppose at some iteration in Algorithm 4.2, the innermost variable is v, and there is a node N that has been split on v. Then, the purpose of the merge operation on N is to remove N from the SRT such that the new SRT still solves the same QBF problem, and the subtree of N has quantified v out explicitly. Thus, a simple way of performing a merge operation is to simply apply the operators bottom up on the subtree of N. We call this, the folding merge (figure 4.3), since it folds the entire subtree into a leaf.

```
void folding_merge (Node N)
{
    if N is leaf then
        return;
    end if
    folding_merge(N.right_child());
    folding_merge(N.left_child());
    if N needs to be packed then
        N = (v∨ N.left_child() ) ∧ (¬v∨ N.right_child() );
    else            //N needs to be quanted op = (v is universal ?  ∧ :  ∨);
        k = N.splitvar();
        t = N.innermost_ancestor();
        N = Q x_{t+1} ... x_{k-1} [N.left_child() op N.right_child()];
    end if
}
```

Figure 4.3: A recursive definition of Folding Merge

The disadvantage of this method is that all previous splits are merged into one and the modularity of the problem is lost in one go. To elaborate, suppose for example a factor graph has some very specific variables which when removed give smooth conversion to an acyclic graph. In such a case (which is not very rare), almost every existential quantification would lead to splits on those variables. If we could find a way which would do the merge without folding up the entire tree, it would avoid repeated splitting and merging on variables.

The unfolding merge is one such method, which works under the assumption that the variable being merged upon is the innermost variable used in any split. This assumption works well for QBF_solve since when eliminating a variable v, all variables inner to v have already been eliminated from the SRT during previous split operations. In order to see how this works, suppose we have two leaf nodes L_1 and L_2 in the subtree with the same set of assignments except on v. I.e., they have all other assignments same, but are on opposite sides on N, and the functions stored in them are $\phi_{\mathcal{A},v}$ and $\phi_{\mathcal{A},\neg v}$. Then, $\psi_{\mathcal{A}} \equiv \phi_{\mathcal{A},v}\ op\ \phi_{\mathcal{A},\neg v}$, where ψ is $\mathcal{Q}_v v\ \phi$ and op is $\wedge$ or $\vee$ depending on whether $\mathcal{Q}_v$ is universal or existential. The unfolding merge uses this principle by finding matching leaves, splitting wherever necessary, and "merges" them

```
void unfolding_merge (Node N)
{
   Let C_1 and C_2 be the two children;
   Let V_i be the split vars in C_i for i = 1,2;
   Expand C_1 and C_2 completely on all variables in V_1 ∪ V_2;
   Synchronize the split ordering of C_1 and C_2;
   C = C_1 op C_2;
   N = C;
}
```

Figure 4.4: A definition of Unfolding Merge

into leaf nodes using *op*. In a sense, it unfolds the two subtrees of v and merges them into one subtree in its place.

A simple pseudo code for *unfolding_merge* is given in figure 4.4. The code creates complete trees from the split variable set, synchronizes the two trees and merges them. By synchronizing the two trees, we mean that the order of split variables in the two trees is the same. This is not very hard in a complete tree, since a split node on v with both children split nodes on w can be easily swapped (into a split node on w with two children on v) by just exchanging two grand-children nodes. One might also think of keeping the SRT always synchronized in a particular order, but that is not advisable since synchronizing incomplete trees might require additional splits which are otherwise unnecessary. Also, from another perspective, allowing the SRT in any order and synchronizing it whenever required is a lazy synchronization, and hence avoids unnecessary computations.

The obvious disadvantage of unfolding merge, not only for this simplistic algorithm but in general, is that it may cause too many splits. In the simplistic algorithm given in figure 4.4, for left and right children of size n_1 and n_2, in the worst case we might end up with a resultant subtree of size $2^{n_1+n_2}$. Instead, it would be better to have something intermediate of these two algorithms, which folds some nodes, and unfolds others.

One alternative would be to unfold merge up to a certain level and unfold everything after that. In our implementation, we split on nodes which already appear in the assignment sets of many leaves, and pack up (since v is the innermost, all nodes in the subtrees are pack nodes) nodes which do not occur in the assignment sets of many leaves. Currently, we accept a threshold value from the user to check whether the number of leaves is sufficient or not.

4.4 Deductions

Consider a clause in a CNF with only universally quantified variables. Then, we know that the QBF as a whole is false, since as we remove the universally quantified variables, the clause will become an empty clause and there will be no way to satisfy it.

Similarly, when a clause has only one existentially quantified variable v with all the other universal variables inside of v in the quantifier ordering, then we can be certain that for the QBF to be true, v should be fixed to take the value as given in the clause. For example, in the QBF

$$\forall x \exists v \forall y \exists w \ (x \vee v \vee y) \wedge (v \vee y \vee \neg w) \wedge (\neg v \vee y)$$

we are certain that v should take the value $\perp$ because of the clause $(\neg v \vee y)$. This and other such deductions are obtainable by simply looking at the CNF structure, but go a long way in eliminating

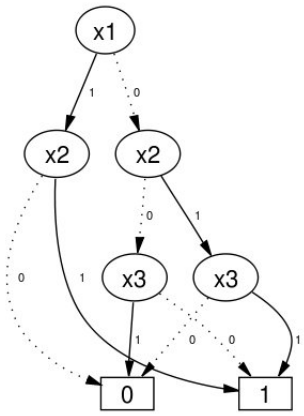

Figure 4.5: A sample BDD

variables from clauses. These form a major backbone of many current state of the art QBF solvers such as quaffle.

Unfortunately, though CNFs are very convenient in terms of presenting these deductions, they are memory inefficient compared to BDDs. Also, current state of the art BDD implementations are empirically much faster in performing other operations such as conjunctions, disjunctions, existential quantification, etc. Hence, we propose a method of performing the above mentioned deductions on BDDs, rather than CNFs.

A BDD can be converted into a CNF format by listing the paths from the root to the leaves. Each path that leads to False is a conjunctive clause in the DNF form of the negation of the BDD. In other words, we can get a clause in the CNF of a BDD by negating the literals in a path from the root to False. For example, the negation of the sample BDD in figure 4.5 taken from the Wikipedia page on BDDs, can be converted to the DNF

$$(x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3) \vee (\neg x_1 \wedge x_2 \wedge \neg x_3)$$

And hence, the given BDD can be converted to the CNF

$$(\neg x_1 \vee x_2) \wedge (x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_3)$$

The method that we propose computes the deductions using these paths implicitly, without converting the BDD into a CNF explicitly. In each path, we see if there is at most one existential variable, and it is the outermost of all variables in that path. Also, posing this as a recursive algorithm would allow us to use the Cudd cache, and of course, would lead to cleaner code.

For a first algorithm, each call to `bdd_deduce` returns a set of BDD variables. An existential variable v in the set means that there is a path from that BDD to zero such that v is the only existential variable in that path, and the outermost of all variables in the path. A universal variable v means that there is some path with only universal variables, and v is the outermost universal variable. The algorithm is as follows :

```
Set bdd_deduce (BDD b, Quant Q)
{
   if b is True then
      return NULL;
```

```
    else if b is False then
        return empty set;
    end if
    b1 = bdd_deduce(b.then_child(), q);
    b2 = bdd_deduce(b.else_child(), q);
    v = b.variable();
    if v is existential then
        remove all existential variables from b1 and b2;
        remove all universal variables which are outside of v in Q;
        ans = empty_set();
        if b1 is non-null and non-empty or b.then_child is False then
            ans = ans ∪ v;
        end if
        if b2 is non-null and non-empty or b.else_child is False then
            ans = ans ∪ ¬v;
        end if
        return ans;
    end if
    // v is universal
    Set b12 = b1 ∪ b2;
    for each variable w in b12 do
        if w is universal and w is inside of v in Q then
            b12 = (b12 \ {w}) ∪ {v};
        else if w is existential and is inside of v in Q then
            b12 = (b12 \ {w}) ∪ {v};
        end if
    end for
    if b.then_child() is False then
        b12 = b12 ∪ v;
    end if
    if b.else_child() is False then
        b12 = b12 ∪ ¬v;
    end if
    return b12;
}
```

The above algorithm can be easily incorporated into the Cudd framework. Sets can be replaced by BDD cubes (conjunction of variables), and hence the deduce operation can be stored into the Cudd cache. However, since a set can contain both a variable and its negation, using a BDD cube directly will not work as the cube will become false. Instead, we can make this work by keeping extra variables that represent the negation of each already existing variable, and use them instead of negations.

An inherent disadvantage of this algorithm is that most of the paths are lost when we encounter a existential variable. For example, if we have a existential variable on the top of a BDD, we are left with at most two deductions. Since the CNF form of a formula is non-unique, other CNF forms may actually come up with much better sets of deductions, and it seems that this is a particularly bad way of converting a BDD into a CNF. One way of overcoming this problem is to use resolution to generate new

clauses and deduce from them. Fortunately, it turns out that resolution can be adopted very smoothly in the bottom-up clause generation, owing to the fact that a variable occurs as opposite literals in its left branch and right branch. Thus, the disjunction of a clause from the left branch and another from the right branch of a BDD leads to a new clause in the CNF of the BDD. We can use this fact and merge each clause on the right of a BDD with each clause on the left, and come up with new clauses which do not even have the BDD variable. However, there is a seemingly small catch which disallows us from using just the outermost variable from the path as in the previous algorithm - if two clauses being resolved on a variable v have a variable w (other than v) occurring as opposite literals in the two clauses, then the two clauses cannot be resolved. For example, $(v \vee w \vee x \vee z)$ and $(\neg v \vee \neg w \vee x \vee y)$ cannot be resolved on the variable v because that would lead to $(w \vee \neg w \vee x \vee y \vee z) \equiv \top$. Thus, if we were to enable such resolution in the above given algorithm, we would need to store each and every literal of each path, rather than just the outermost one. The new algorithm, hence, returns sets of clauses rather than sets of variables, and these clauses can then be scanned to check for deductions.

```
Set bdd_deduce_resolution (BDD b, Quant Q)
{
   if b is True then
      return NULL;
   else if b is False then
      return empty set;
   end if
   b1 = bdd_deduce_resolve(b.else_child(), q);
   b2 = bdd_deduce_resolve(b.then_child(), q);
   b12 = cross_product(b1, b2);
   v = b.variable();
   b1.append_to_each_clause(v);
   b2.append_to_each_clause(¬v);
   return b1 ∪ b2 ∪ b12;
}
```

Clearly, this algorithm has a large memory and computation overhead, and an intermediate algorithm between `bdd_deduce` and `bdd_deduce_resolution` is on the drawing board.

Chapter 5

Experiments

For our experiments we ran the factor graph algorithm on the standard Rintanen suite of qdimacs files. The machine had two 1.73 GHz Intel Pentium Dual CPUs and all experiments were allowed upto 2.5 Gbs of RAM.

Each qdimacs file had several alternating universal and existential quantifier blocks, followed by a matrix. For each existential block, clauses from the (original) matrix with at least one quantified variable were filtered out, and turned into a factor graph, with each variable and each clause as an individual node. Then, all variables which were not to be quantified out were merged into a single function node, so that the projection could be taken on them. The resultant graph was then made acyclic with a lambda (estimated Largest Support Set) of 85. Whenever a split was required, factor graphs on both sides of the split were made acyclic and converged upon recursively, and the two final sub results were disjoined as an estimate of the time taken to combine them. For each quantifier block, a time of 2 hours was given, and a limit of 40 splits was given to avoid an exponential blowup. Each quantifier block was also done starting with a fresh Cudd Cache, so that essentially each block, rather than each qdimacs file was an individual problem instance. Reusing nodes and edges while doing and undoing splits was done, but reusing of messages was not implemented.

In all there were about 66 quantifier blocks, out of which 28 computed the result successfully, 33 were terminated due to excessive splitting, and 5 were timed out. The successful, aborted, and timed out experiments are given in tables 5.1, 5.2 and 5.3 respectively. In all these tables, the block size gives an estimate of the number of nodes in the factor graph, since all the other variables are merged into one node. The number of peak nodes and number of live peak nodes are Cudd statistics for keeping an eye on the peak memory requirement. The average depth is $\log_2$ of the number of leaves of the split tree that were solved to get the answer.

In the set of successful runs in table 5.1, there is an apparent anomaly. The factor graph is unbelievably fast for most of the large sets, but is much slower for a few graphs with around 100 nodes. The explanation for this is that the large graphs that went through had inherent structure in them, and did not require very large merged nodes for them to become acyclic. Indeed, the blocks with large memory and time requirements are the only ones which required any splits. Also, this hypothesis is more supported by the fact that the entries in table 5.2 begin from 144 nodes, which is right after where the above mentioned badly structured instances ended (112 nodes).

From table 5.2 we get a rough idea of the complexity of factor graph manipulations. Theoretically, the current inefficient `make_acyclic` algorithm may have a worst case complexity of $\mathcal{O}(n^3)$ where n is the number of nodes in the graph. Plotting the time taken versus the number of nodes in the graph for those entries which do not have even a single convergence (average depth = 0) gives us the graph in 5.1.

S. no.	Blocksize	Total vars	Avg. depth	Peak nodes	Peak live nodes	Time taken(sec)
1.	9	25	0	1022	158	0.01
2.	18	25	0	2044	269	0.01
3.	24	34	0	3066	476	0.01
4.	36	170	0	34748	2622	0.37
5.	54	108	0	10220	509	0.02
6.	54	153	0	31682	4371	1.01
7.	54	192	0	64386	7462	2.16
8.	72	144	0	18396	671	0.04
9.	77	214	1	19310690	16327675	1930.97
10.	84	234	2	74913622	74907526	7149.81
11.	96	636	0	415954	415954	9.36
12.	104	292	5	25064550	24349970	1735.03
13.	112	315	5	25072726	24350095	1719.57
14.	240	480	0	202356	2138	0.38
15.	270	451	0	58254	2975	1.27
16.	288	576	0	290248	2554	0.5
17.	288	576	0	290248	2640	0.38
18.	338	676	0	400624	3094	0.61
19.	392	784	0	538594	3584	0.77
20.	396	661	0	119574	4621	4.26
21.	450	900	0	709268	4110	0.99
22.	512	1024	0	917756	4672	1.27
23.	578	1156	0	1169168	5270	1.7
24.	648	1296	0	1469636	5904	2.14
25.	722	1444	0	1606584	6574	2.79
26.	800	1600	0	1473724	7280	3.26
27.	882	1764	0	1429778	8022	4.05
28.	968	1936	0	1504384	8800	4.98

Table 5.1: Successful executions of the factor graph algorithm

Although the graph on the whole has many anomalies, a small quadratic/cubic trend can be seen towards the end points. After filtering some more points, we can see a smooth trend in figure 5.2. Upon taking a plot of the $\log_{10}$ of the two axes (figure 5.3), we see that the graph is almost a straight line with slope around 2.4. Thus, the asymptotic empirical complexity of the `make_acyclic` algorithm seems to be some where around $\mathcal{O}(n^{2.4})$.

An important inference that we can make from table 5.3 is that, if left on their own, some of these graphs may take a very long time to converge.

In all, there were 26145 calls to the message passing algorithm, out of which 19512 converged in 3 iterations, 15 converged in 1 iteration and 6618 converged in 4 iterations. Thus, empirically it seems that the theoretical upper bound given in this dissertation is much looser than reality.

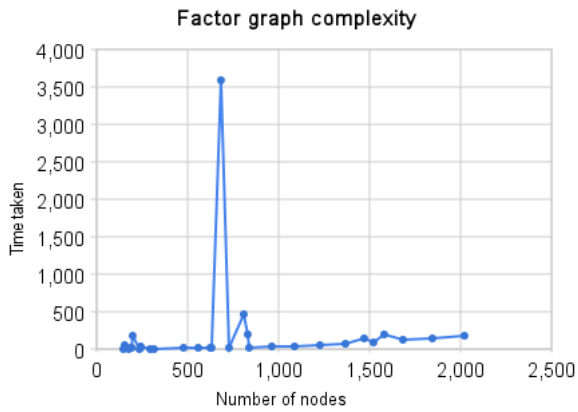

Figure 5.1: Plot of time taken to make a graph acyclic vs number of nodes

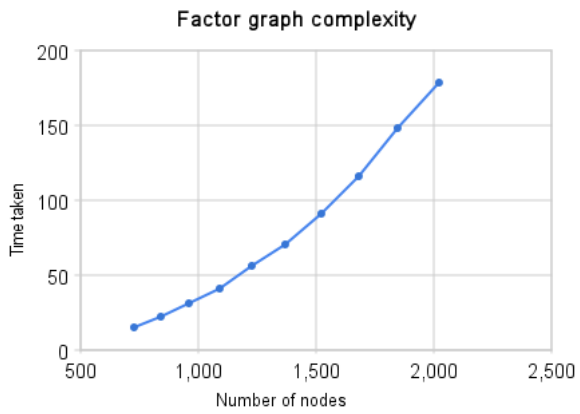

Figure 5.2: Filtered asymptotic graph

Figure 5.3: Plot of time taken to make a graph acyclic vs number of nodes

S. no.	Blocksize	Total vars	Avg. depth	Expected depth	Time taken(sec)
1.	144	202	0	41	1.39
2.	156	288	0	41	59.56
3.	180	256	0	41	2.63
4.	189	247	0	41	15.05
5.	192	286	0	41	22.12
6.	196	328	0	41	175.75
7.	220	630	7.89	17	598.87
8.	234	324	0	41	1.87
9.	242	282	0	41	10.02
10.	246	304	0	41	38.79
11.	272	532	3.90689	22	204.52
12.	292	391	0	41	5.26
13.	315	422	0	41	7.52
14.	480	727	0	41	15.82
15.	544	1584	7.2854	38	116.5
16.	560	855	0	41	19.68
17.	625	925	0	41	10.91
18.	630	843	0	41	20.09
19.	687	838	0	41	3592.76
20.	729	1080	0	41	15.46
21.	812	915	0	41	473.06
22.	828	1003	0	41	188.75
23.	841	1247	0	41	21.91
24.	961	1426	0	41	30.89
25.	1089	1617	0	41	41.37
26.	1225	1820	0	41	55.93
27.	1369	2035	0	41	70.75
28.	1474	1731	0	41	139.15
29.	1521	2262	0	41	90.82
30.	1584	2115	0	41	201.36
31.	1681	2501	0	41	116.51
32.	1849	2752	0	41	148.35
33.	2025	3015	0	41	178.28

Table 5.2: Executions which were aborted due to excessive splits

S. no.	Blocksize	Total vars	Depth finished	Estimated depth
1.	90	174	0	11
2.	128	194	0	0
3.	128	216	10	20
4.	144	591	5	24
5.	500	779	0	0

Table 5.3: Executions which were aborted due to a time restriction of 2 hours

Chapter 6

Future work

Some important theoretical gaps to be covered are as follows:

- *Bound in over-approximation* It is yet to be seen if there can be any bound in the over approximation produced due to cycles in the factor graph. Currently, we haven't found any such bound or any condition in the expression which can guarantee some bound.
- *Upper bound on the number of iterations* The theoretical upper bound given in this document does not match experimental results. In practice, the number of iterations is almost always 3 and never more than 4.
- *Reusing of messages* It would be very difficult indeed if every time a variable is split, the problem breaks up into twice the size. Instead, experience suggests that reusing of computation from one branch into another might go a long way in avoiding the worst case performance. We have explored somewhat in this regard, but the estimate on number of iterations taken by the original method needs to be brought down for efficient reuse (see section 2.7).
- *Deductions in BDDs* While doing deductions without resolution may lead to very few deductions, enabling resolution has much higher memory (and computational) requirements. A new intermediate framework is yet to be devised so that efficiency and output are balanced. Also, we have looked at only unit clauses, where as there are many known deductions from the CNF format, and possibly many in the BDD format.
- *Unfolding merge* The current algorithm in unfolding merge unfolds the tree entirely, which might be too large. Instead, an algorithm which only does the minimal number of splits required would be better.

There are also many things which would be better verified experimentally, and many utilities that are yet to be implemented :

- *Leveraging quantifier scheduling* In the algorithm that leverages quantifier scheduling with message passing, it would be worth investigating whether using exact computation as much as possible indeed helps or not.
- *Symbolic model checking* One of the major contributions of this thesis is about how cyclic factor graphs give over approximations, and how it is very useful in the domain of symbolic model checking. Experimentally however, we are yet to verify how this method works out. Also, such an experiment would give a definitive answers on how bad can the over-approximations due to cycles be in practice.

- *Split Record Table* Though an implementation of SRTs is underway, there is still many bugs that need to be tackled before we have a full fledges QBF solver. Once we have that, we can then compare our solver with other standard solvers and see how well do factor graphs work in action.

- *Making the graph acyclic* The current implemented algorithm starts the BFS exploration for cycles from scratch every time two nodes are merged. Removal of a single cycle may take multiple BFSs/ Instead, one cycle can be completely removed in one iteration of `factor_graph_make_acyclic`, and exploration can be continued from the last frontier behind the cycle. This would lead to fewer and faster iterations than the current $\mathcal{O}(n^{2.4})$.

- *Reusing of messages* Currently such a feature has not been implemented, although intuitively it seems very promising in making the exponential blow-up of splitting a little less severe. It would be interesting to empirically observe how much of a difference it makes.

Bibliography

S. B. Akers. Binary decision diagrams. *IEEE Trans. Computers*, 27(6):509–516, 1978.

Annervaz. Image computation using the sum product algorithm. Master's thesis, Indian Institute of Technology, Bombay, 2009.

B. Bollig and I. Wegener. Improving the variable ordering of obdds is np-complete. *IEEE Transactions on Computers*, 45(9):993–1002, 1996. ISSN 0018-9340.

A. Braunstein, M. Mezard, and R. Zecchina. Survey propagation: an algorithm for satisfiability. *Random Structures and Algorithms*, 27:201, 2005. URL `http://www.citebase.org/abstract?id=oai:arXiv.org:cs/0212002`.

J. R. Burch, E. M. Clarke, D. E. Long, K. L. Mcmillan, and D. Dill. Symbolic model checking for sequential circuit verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 13:401–424, 1993.

M. Järvisalo, A. Biere, and M. Heule. Blocked clause elimination. In *TACAS*, pages 129–144, 2010.

F. Kschischang, S. Member, B. J. Frey, and H. A. Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47:498–519, 2001.

L. Zhang and S. Malik. Towards a symmetric treatment of satisfaction and conflicts in quantified boolean formula evaluation. pages 200–215. Springer, 2002.